

Revisiting the Class Imbalance Issue in Software Defect Prediction

Md. Fahimuzzman Sohan¹, Md Alamgir Kabir^{1,3}, Md. Ismail Jabiullah²,
Sheikh Shah Mohammad Motiur Rahman¹

¹Department of Software Engineering, Daffodil International University, Dhaka, Bangladesh

²Department of Computer Science Engineering, Daffodil International University, Dhaka, Bangladesh

³Department of Computer Science, City University of Hong Kong

Email: {sohan35-1284, alamgir.swe, drismail.cse, motiur.swe}@diu.edu.bd

Abstract—Software defect prediction is related to the testing area of software industry. Several methods have been developed for the prediction of bugs in software source codes. The objective of this study is to find the inconsistency of performance between imbalances and balance data set and to find the distinction of performance between single classifier and aggregate classifier (voting). In this investigation, eight publicly available data sets have collected, also seven algorithms and hard voting are used for finding precision, recall and F-1 score to predict software defect. In these collected data, two sets are almost balanced. For this investigation, these balanced data sets have converted into imbalanced sets as average non-defective and defective ratio of the other 6 data sets. The experiment result shows that performance of the two balanced data sets is lower than other six sets. After conversion of two data sets, the performance has increased as like as other six data sets. Another observation is the performance metric that shows the results of precision, recall and F1-score for voting are 0.92, 0.84 and 0.87 respectively, which are better than other single classifier. This study has been able to shows that- imbalance of non-defective and defective classes have a big impact on software defect prediction and the voting is the best performer among the classifiers.

Keywords— *Software defect prediction, machine learning algorithm, non-defective and defective class, software metric.*

I. INTRODUCTION

Developers are fighting constantly with defects from the beginning of the software history. The defect has important contribution and plays a valuable role in the time of releasing software. Software defect may be detected before the software release or after the release. But the expectation of developers is to provide defect free software to their clients. Therefore, identifying the software defect before releasing makes a good advantage for the developers. Here, the challenging task is to find the defect prone area or defective module of software. If defective modules are detected before the release of software then it could be easily prevented and repaired by the developer. This early detection has big impact on software quality; moreover it can reduce cost and time [1]-[3]. However, in software defect prediction, a class of a project will be non-defective or defective. It depends on software metrics, such as Line of Code (LOC), Number of Public Methods (NPM), Average Method Complexity (AMC) etc. Every metric has a separate method to find the metric's value, based on these values non-defective and defective classes are identified in testing phase [4]-[6]. A defect prediction data set is prepared from the number of non-defective and defective classes of a project. Researchers have used lot of balanced and imbalanced data sets to predict

the defect. The performance of defect prediction model by using balanced and imbalanced data sets make big impact on software testing. To prepare the data set, the different number of metrics has been considered. Some data sets are based on 19 metrics, some are 40, 42, 63 and 209 [7], [8]. Maximum data sets are public and these have been used by many researchers in this area [9], and also many software defect prediction public data sets are available in ZENODO¹, PROMISE² repository. The defect prediction data set has prepared from historical data. Over the years, many overwhelming machine learning approaches have been developed and used for software defect prediction [10]. In defect prediction approach, the data set is divided into train and test part. The classifier is learned from the training data and this learning is used to predict the defect of the test data [7]. Many machine learning techniques have been used by incorporating the learning approaches such as supervised learning [2], [11], semi-supervised learning [7], [12], imbalance learning [10], [13]. Moreover, these approaches have dominated the task, software defect prediction. Based on our literature, one of the approaches is Wahono [9] investigation. His investigation shows that the classifiers- Naïve Bayes (NB), Decision Tree (DT), Neural Network (NN) and Random Forest (RF) are performed well to classify and identify the defect prone modules in source codes.

The goal of this study is performed seven classifiers and voting algorithms on eight publicly available data sets. Moreover, for more investigation two data sets have re-arranged out of eight sets by their non-defective and defective class ratio. This re-arranging operation gives an interesting output: traditional imbalanced data sets give higher accuracy than balanced (almost) data sets, but after changing the balance mode to imbalance ratio the accuracy has increased. After that, the performance has measured by using precision, recall and F1-score. In this experiment, every data set has investigated with the every single algorithm and Voting. Thereby the average performance has estimated.

The rest of the paper is organized as follows: Section II represents the related work. The experiment detail and methodology are presented in the next section. Result analysis is discussed in section IV. Section V concludes the paper.

¹ <https://zenodo.org/communities/seacraft/search?page=1&size=20&q=&keywords=defect>

² <http://promise.site.uottawa.ca/SERepository/datasets-page.html>

II. RELATED WORK

This Static and dynamic defect predictions are two major software defect prediction technology. The typical static technique has some steps; in [14], Wang et al. have shown it briefly. In [15], Petrić et al. have been discussed on 3 questions in their research paper. They have shown the answers against their questions like: stacking ensembles can improve the performance, stacking ensemble does not require many base classifiers, Naïve Bayes, SMO and kNN combine together for stacking ensemble gives performance. Also, by many ways researchers have been used ensemble learning (aggregated classifiers) and it performs well; such as Bagging [16], Voting [17], [18]. In [19], Rodriguez et al. have been used well-known NASA data sets. They have found imbalanced data sets give the correct classification of the minority (defective) class.

Sometimes size of a data set may affect on the performance. In [20], Abaei & Selamat have been used 4 NASA data sets; also the data sets have different size and different number of non-defective and defective data. They have found Random Forest (RF) and Naïve Bayes (NB) provide best performance for large data sets and small data sets respectively. Artificial Neural Network (ANN) have been used in many research paper. One of recent paper has shows a comparison between ANN and Support Vector Machine (SVM) [21], ANN gives better result in term of accuracy and specificity but under the same data sets SVM is better than ANN in term of recall. In [22], Peng et al. have contributed an empirical study with 13 classifiers, 11 public data sets and various performance measures. Their experiment have implemented in WEKA, and the result shows top three performer classifiers are SVM, C4.5 and k-NN.

Recently in this area, a lot of hullabaloo has pointed into balance and imbalance data sets software defect prediction. Many research papers have investigated on class imbalance issue. In [10], Song et al. have pointed that in software defect prediction, imbalance learning only suitable for moderate or highly imbalanced data sets. In [13], Bennin et al. have worked with highly imbalanced datasets and show that their MAHAKIL improves the predation performance. In [23], Chen et al. they have selected 9 highly unbalanced data sets for their experiment and compared the result with other state-of-the-art methods. In [34], Benin et al. have worked on resample data to identify the statistical and practical importance for software defect prediction model. In [35], Wang & Yao have investigated on balance, G-mean, and AUC. They have showed the result of balance data set is better than imbalance learning except Naïve Bayes classifier. They have also proposed a new version of AdaBoost, it gives better prediction rate than normal AdaBoost.

III. METHODOLOGY

A. Data sets

For this investigation, 8 releases of 4 open source project's data have been collected from Jean Petrić, this data sets they have been used in their research paper [15]. In another research paper [4], from Jureczko & Madeyski detail description have discussed about the data sets. 19 metrics (CK metrics)³ have considered for this 8 data sets and this metrics are prepared by size and complexity.

TABLE I. DATA SET METRICS

	Metrics
1	Weighted Methods per Class (WMC)
2	Depth of Inheritance Tree (DIT)
3	Number of Children (NOC)
4	Coupling between Object Classes (CBO)
5	Response for a Class (RFC)
6	Lack of Cohesion in Methods (LCOM)
7	Afferent couplings (Ca)
8	Efferent couplings (Ce)
9	Number of Public Methods (NPM)
10	Lack of cohesion in methods (LCOM3)
11	Lines of Code (LOC)
12	Data Access Metric (DAM)
13	Measure of Aggregation (MOA)
14	Measure of Functional Abstraction (MFA)
15	Cohesion Among Methods of Class (CAM)
16	Inheritance Coupling (IC)
17	Coupling Between Methods (CBM)
18	Average Method Complexity (AMC)
19	McCabe's Cyclomatic Complexity (CC)

Table 1 provides the name of 19 metrics. By the metrics the decision have been taken either a class is defective or non-defective in the period of testing. Table 2 represents details about the data set configuration. One thing that can be identified for first 6 data sets from this Table is the percentage gape of non-defective and defective classes are negligible if anyone compares with next two data sets. But later on, non-defective and defective classes are almost equal for Xalan-2.5 and Xalan-2.6 data sets.

TABLE II. SUMMARY OF 8 DATA SETS

Data Set	Class	Non-defective (N)	Percentage of N	Defective(Y)	Percentage of Y
ant-1.5	292	260	89.04%	32	10.96%
ant-1.6	350	258	73.71%	92	26.29%
ant-1.7	742	576	77.63%	166	22.37%
jedit-4.1	312	233	74.68%	79	25.32%
jedit-4.2	367	319	86.92%	48	13.08%
tomcat	852	775	90.96%	77	9.04%
xalan-2.5	797	410	51.44%	387	48.56%
xalan-2.6	880	469	53.3%	411	46.7%

For detail investigation, the Xalan-2.5 and Xalan-2.6 data sets have newly stored like first 6 data sets. The average non-defective and defective classes for first 6 data sets was 82% and 18% respectively. To find a suitable ratio for Xalan-2.5 and Xalan-2.6, the 82% and 18% have been considered as standard ratio for making 2 new data sets, which are given in Table 3.

TABLE III. SUMMARY OF 2 NEW DATA SETS

Data set	Class	Non-defective(N)	Percentage of N	Defective(Y)	Percentage of Y
New xalan-2.5	484	410	82%	74	18%
New xalan-2.6	552	469	82%	84	18%

³ http://gromit.iar.pwr.wroc.pl/p_inf/ckjm/

B. Used Algorithms

Machine learning algorithm has an important role in this area. The learning algorithms are used to learn the training data. Wahono [9] has a review paper; it has shown 7 most used algorithms including NB, DT, NN, RF and k-NN in this area. In software defect prediction, Naïve Bayes (NB) performs well, also it is common and top used algorithm. Decision Tree (DT) is popular one and second top used algorithm in this field according to his investigation. Neural Network (NN) has used in several investigations and gives good accuracy [21], [24], [25].

In this study, 7 machine learning algorithms have been considered: Decision Tree (DT), Random Forest (RF), Ada Boost (AdB), Gradient boost (GB), Naïve Bayes (NB), k-Nearest Neighbors (k-NN) and Artificial Neural Network (ANN). On the other hand, hard voting classifier is used as an aggregated classifier. In this voting, Logistic Regression (LR), Decision Tree (DT) and Support Vector Machine (SVM) are used together.

C. Performance Measure

The concept of machine learning is learn to the machine as like human being, and performance measure is the report of how accurately learn the machine. Performance Measures shows the statistical view of an experiment's result. This performance depends on the classifier algorithm that is used in the experiment and for different algorithm the result can be different [26].

precision, recall and F1-score have been used to evaluate the experiment and these are widely used evolution metrics in this area [27]-[29]. Here, Confusion Matrix is needed for finding precision, recall and F1-score. Confusion Matrix represents: True Positive (TP), False Positive (FP), False Negative (FN), and True Negative (TN); from these 4 criterions the 3 evaluation metrics have been calculated.

TABLE IV. CONFUSION MATRIX FOR THIS STUDY

Criteria	Action
TP	Predict defective class as defective
FP	Predict non-defective class as defective
FN	Predict defective class as no-defective
TN	Predict non-defective class as non-defective

Table 4 represents how TP, FP, FN and TN work in this experiment. In Table 5, the mathematical formulas have been given to find precision, recall and F1-score [29], [30].

TABLE V. MATHEMATICAL REPRESENTATION OF PRECISION, RECALL AND F1-SCORE

Evaluation Metrics	Formula
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1-score	$\frac{2 \times Precision \times Recall}{Precision + Recall}$

D. Experiment Setup

This section represents the required procedure of the experiment. Eight data sets have been considered in this study, and for detail investigation 2 data sets have been newly prepared from Xalan-2.5 and Xalan-2.6. Data set splitting is an important component of this area. In many experiments [31, 32], they supplied training and test set individually into their simulations and some of them split the data set into training and test set in the time of simulations [15], [33].

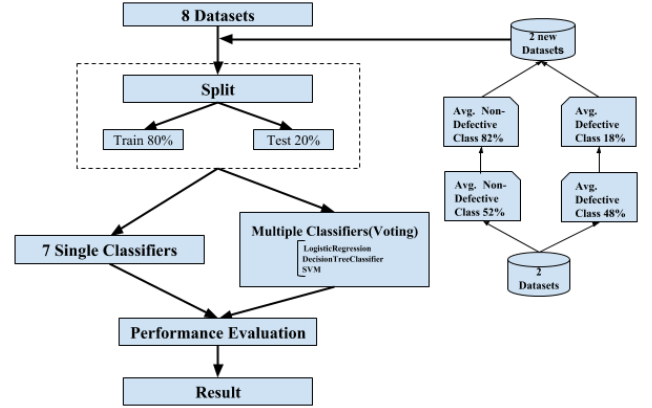


Fig. 1. Data Sets Rearranging and Experiment Procedure

Figure 1 shows every data set has spited into 80% train set and 20% test. That means 80% class of a data set has trained and from this trained part, the rest data have tested. After that, 7 single classifiers and hard voting are performed. After successfully executed these algorithms a classification report is given. From this report the precision, recall and F1-score are collected. This experiment represents two observations; one is performance comparison of Xalan-2.5 and Xalan-2.6 with new two datasets and single classifiers with Voting.

IV. RESULT ANALYSIS

This experiment represents two observations; one is the performance comparison of Xalan-2.5 and Xalan-2.6 with new Xalan-2.5 and Xalan-2.6, another is the performance comparison of 7 single classifiers and voting. For all the data sets, 80% data are trained and 20% data have learned from the trained data, the 20% tested data result have discussed here. The simulation of this study has been done by Jupyter Notebook. This section has divided into 2 parts: Case Study #1 and Case Study #2, those are given bellow.

A. Case Study #1

In Table 6, precision, recall and F1-score have given for Xalan-2.5 and Xalan-2.6 dataset. For these two data sets the non-defective and defective classes are almost equal. The average performance of these two data sets according to 7 single algorithms and voting are highlighted with boldfaced print in Table 6.

TABLE VI. RESULT OF XALAN-2.5 AND XALAN-2.6 DATA SETS BEFORE RE-ARRANGE

	Dataset	DT	RF	AdB	GB	NB	KNN	ANN	Voting
Precision	xalan-2.5	0.67	0.71	0.67	0.64	0.79	0.62	0.65	0.76
	xalan-2.6	0.66	0.78	0.8	0.76	0.83	0.7	0.7	0.83
	Average	0.67	0.75	0.74	0.7	0.81	0.66	0.68	0.8
Recall	xalan-2.5	0.64	0.68	0.63	0.32	0.59	0.62	0.56	0.53
	xalan-2.6	0.66	0.76	0.79	0.75	0.74	0.7	0.69	0.68
	Average	0.65	0.72	0.71	0.54	0.67	0.66	0.63	0.61
F1-score	xalan-2.5	0.64	0.68	0.64	0.63	0.65	0.62	0.59	0.58
	xalan-2.6	0.66	0.76	0.79	0.75	0.75	0.7	0.69	0.7
	Average	0.65	0.72	0.72	0.69	0.7	0.66	0.64	0.64

Table 7, which represent the performance of new Xalan-2.5 and Xalan-2.6 data sets. These two data sets are made by following the ratio of previous 6 data sets. In previous 6 data sets majority class was non-defective and minority class was defective. But, Xalan-2.5 and Xalan-2.6 are not flowing this ratio (information has given in Table 2). Interesting thing is Xalan-2.5 and Xalan-2.6 give lower result (Table 6 indicates that) than the other 6 data sets (Table 8, 9 and 10 indicate

that). After that, from the 2 Xalan data sets 2 new sets have prepared under the average non-defective and defective ratio (Table 3) of previous 6 data sets. In Table 6 and 7, average precision, recall and F1-score value for 7 algorithms and voting have been given according to Xalan-2.5, Xalan-2.6 and new Xalan-2.5, Xalan-2.6 with boldfaced print. After re-arrange the data sets the performance has increased which are given in Table 7.

TABLE VII. RESULT OF NEW XALAN-2.5 AND XALAN-2.6

	Dataset	DT	RF	AdB	GB	NB	KNN	ANN	Voting
Precision	xalan-2.5	0.78	0.91	0.89	0.88	0.86	0.84	0.72	0.94
	xalan-2.6	0.82	0.9	0.83	0.87	0.87	0.83	0.86	0.9
	Average	0.8	0.91	0.86	0.88	0.87	0.84	0.79	0.92
Recall	xalan-2.5	0.77	0.88	0.86	0.86	0.79	0.84	0.73	0.79
	xalan-2.6	0.83	0.87	0.83	0.87	0.83	0.81	0.78	0.83
	Average	0.8	0.88	0.85	0.87	0.81	0.83	0.76	0.81
F1-score	xalan-2.5	0.77	0.89	0.87	0.87	0.82	0.84	0.71	0.85
	xalan-2.6	0.82	0.87	0.83	0.87	0.84	0.82	0.81	0.85
	Average	0.8	0.88	0.85	0.87	0.83	0.83	0.76	0.85

B. Case Study #2

Case study #2, which presents the precision, recall and F1-score for all the data sets under the 7 classifiers and voting. Here, new values of Xalan-2.5 and Xalan-2.6 have been used. After taking these values, the average performance measures have shown in Table 8. In precision, recall and F1-score, Random Forest (RF) achieves highest performance (0.91, 0.87 and 0.88 respectively) among the 7 individual classifiers. So, it can be considered that Random Forest (RF) is high performer than other 6 classifier under

these 8 data sets. Table 9 shows the summary of Table 8. The average precision, recall and F1-score according to 8 data sets 7 algorithms are given in Table 9, the average value is 0.85, 0.82 and 0.87 respectively. But the exciting matter is, according to 8 data sets the average precision, recall and F1-score of voting are 0.92, 0.84 and 0.87. This Table shows voting can achieve better performance competitively other single algorithms.

TABLE VIII. AVERAGE PRECISION, RECALL AND F1-SCORE OF 8 DATA SETS ACCORDING TO 7 SINGLE CLASSIFIERS AND VOTING

	Dataset	DT	RF	AdB	GB	NB	KNN	ANN	Voting
Precision	ant-1.5	0.71	0.93	0.87	0.9	0.73	0.91	0.85	0.9
	ant-1.6	0.75	0.9	0.86	0.85	0.82	0.84	0.76	0.92
	ant-1.7	0.75	0.85	0.79	0.82	0.81	0.81	0.77	0.89
	jedit-4.1	0.76	0.86	0.77	0.83	0.9	0.81	0.71	0.9
	jedit-4.2	0.87	0.96	0.98	0.94	0.86	0.89	0.87	0.96
	tomcat	0.84	0.94	0.86	0.92	0.83	0.89	0.77	0.97
	xalan-2.5	0.78	0.91	0.89	0.88	0.86	0.84	0.72	0.94
	xalan-2.6	0.82	0.9	0.83	0.87	0.87	0.83	0.86	0.9
	Average	0.79	0.91	0.86	0.88	0.84	0.85	0.79	0.92
Recall	ant-1.5	0.81	0.93	0.9	0.92	0.75	0.86	0.81	0.92
	ant-1.6	0.76	0.89	0.84	0.81	0.8	0.8	0.7	0.81
	ant-1.7	0.75	0.81	0.78	0.81	0.81	0.81	0.79	0.79
	jedit-4.1	0.71	0.78	0.75	0.76	0.78	0.73	0.67	0.75
	jedit-4.2	0.84	0.86	0.84	0.85	0.88	0.89	0.88	0.86
	tomcat	0.88	0.93	0.88	0.92	0.85	0.88	0.81	0.95
	xalan-2.5	0.77	0.88	0.86	0.86	0.79	0.84	0.73	0.79
	xalan-2.6	0.83	0.87	0.83	0.87	0.83	0.81	0.78	0.83
	Average	0.79	0.87	0.84	0.85	0.81	0.83	0.77	0.84
F1-score	ant-1.5	0.76	0.93	0.88	0.91	0.72	0.88	0.83	0.91
	ant-1.6	0.75	0.89	0.85	0.83	0.81	0.81	0.72	0.85
	ant-1.7	0.75	0.83	0.78	0.81	0.81	0.81	0.77	0.82
	jedit-4.1	0.73	0.8	0.75	0.78	0.82	0.76	0.68	0.79
	jedit-4.2	0.85	0.9	0.89	0.88	0.87	0.89	0.86	0.9
	tomcat	0.86	0.93	0.87	0.92	0.84	0.88	0.79	0.96
	xalan-2.5	0.77	0.89	0.87	0.87	0.82	0.84	0.71	0.85
	xalan-2.6	0.82	0.87	0.83	0.87	0.84	0.82	0.81	0.85
	Average	0.79	0.88	0.84	0.86	0.82	0.84	0.77	0.87

TABLE IX. COMPARISON OF PRECISION, RECALL AND F1-SCORE BETWEEN AVERAGE VALUE OF 7 SINGLE CLASSIFIERS AND VOTING

	DT	RF	AdB	GB	NB	KNN	ANN	Average	Voting
Precision	0.79	0.91	0.86	0.88	0.84	0.85	0.79	0.85	0.92
Recall	0.79	0.87	0.84	0.85	0.81	0.83	0.77	0.82	0.84
F1-score	0.79	0.88	0.84	0.86	0.82	0.84	0.77	0.83	0.87

V. CONCLUSION

Balance-imbalance issue is presence for many traditional software defect data sets, and that's why traditional software defect prediction ways can't be suitable for all times. This study shows that class balance-imbalance issue may effect on the evolution result. On the other side, using one more classifiers (like 3 classifiers) together can give more positive evolution for software defect prediction. In this study, 6 data sets are less balanced than other two; the average ratio of non-defective and defective classes is 82% and 18% respectively. The remaining 2 data sets are more balanced, like the average ratio of non-defective and defective classes are 52% and 48% respectively. At first, the experimental result shows that first 6 data sets give better accuracy than other two data sets under this ratio. After changing this ratio for 2 data sets as previous 6 sets (82% and 18%), the accuracy has increased as like the 6 sets. The average precision, recall and F-1 score of 7 single classifiers have

given in Table 9 with the voting score. In this Table, the average value of precision, recall and F-1 score is 0.85, 0.82 and 0.83; and precision, recall and F-1 score for voting is 0.92, 0.84 and 0.87 respectively. It is observable from here that, comparatively aggregated classifier gives higher accuracy than the single classifiers.

Software defect prediction research area is wide. Further steps are the work will be started on cross project defect prediction; the focus will be generating a suitable model by using a lot of historical data and learning classifiers to ensure the prediction more reliable.

REFERENCES

- [1] D. Bowes, T. Hall, and J. Petrić, "Software defect prediction: do different classifiers find the same defects?," *Software Quality Journal*, vol. 26.2, pp. 525-552, 2018.
- [2] K. Bashir, T. Li, C. W. Yohannese, & Y. Mahama, "Enhancing software defect prediction using supervised-learning based framework," In *Intelligent Systems and Knowledge Engineering*

- (ISKE), 2017 12th International Conference on IEEE, pp. 1-6, November 2017.
- [3] G. Mauša, & T. G. Grbac, "Co-evolutionary multi-population genetic programming for classification in software defect prediction: An empirical case study," *Applied Soft Computing*, vol. 55, pp. 331-351, 2017.
 - [4] M. Jureczko, & L. Madeyski, "Towards identifying software project clusters with regard to defect prediction," In *Proceedings of the 6th International Conference on Predictive Models in Software Engineering*, ACM, pp. 9, September 2010.
 - [5] X. Chen, Y. Zhao, Q. Wang, & Z. Yuan, "MULTI: Multi-objective effort-aware just-in-time software defect prediction," *Information and Software Technology*, vol. 93, pp. 1-13, 2018.
 - [6] S. Huda, K. Liu, M. Abdelrazek, A. Ibrahim, S. Alyahya, H. Al-Dossari, & S. Ahmad, "An ensemble oversampling model for class imbalance problem in software defect prediction." *IEEE access*, vol. 6, pp. 24184-24195, 2018.
 - [7] Z. W. Zhang, X. Y. Jing, & T. J. Wang, "Label propagation based semi-supervised learning for software defect prediction," *Automated Software Engineering*, vol. 24(1), pp. 47-69, 2017.
 - [8] H. Wang, T. M. Khoshgoftar, & A. Napolitano, "A comparative study of ensemble feature selection techniques for software defect prediction," In *Machine Learning and Applications (ICMLA)*, 2010 Ninth International Conference on IEEE, pp. 135-140, December 2010.
 - [9] R. S. Wahono, "A systematic literature review of software defect prediction: research trends, datasets, methods and frameworks," *Journal of Software Engineering*, vol. 1(1), pp. 1-16, 2015.
 - [10] Q. Song, Y. Guo, & M. Shepperd, "A Comprehensive Investigation of the Role of Imbalanced Learning for Software Defect Prediction," *IEEE Transactions on Software Engineering*, 2018.
 - [11] J. Ge, J. Liu, & W. Liu, "Comparative Study on Defect Prediction Algorithms of Supervised Learning Software Based on Imbalanced Classification Data Sets," In 2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), IEEE, pp. 399-406, June 2018.
 - [12] F. Wu, X. Y. Jing, X. Dong, J. Cao, M. Xu, H. Zhang, S. Ying, & B. Xu, "Cross-project and within-project semi-supervised software defect prediction problems study using a unified solution," In *Software Engineering Companion (ICSE-C)*, 2017 IEEE/ACM 39th International Conference on IEEE, pp. 195-197, May 2017.
 - [13] K. E. Bennin, J. Keung, P. Phannachitta, A. Monden, & S. Mensah, "Mahakil: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction," *IEEE Transactions on Software Engineering*, 2017.
 - [14] T. Wang, Z. Zhang, X. Jing, & L. Zhang, "Multiple kernel ensemble learning for software defect prediction," *Automated Software Engineering*, vol. 23(4), pp. 569-590, 2016.
 - [15] J. Petrić, D. Bowes, T. Hall, B. Christianson, & N. Baddoo, "Building an ensemble for software defect prediction based on diversity selection," In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ACM, pp. 46, September 2016.
 - [16] H. Alsawalqah, H. Faris, I. Aljarah, L. Alnemer, & N. Alhindawi, "Hybrid SMOTE-Ensemble Approach for Software Defect Prediction," In *Computer Science On-line Conference*, Springer, Cham, pp. 355-366, April 2017.
 - [17] G. Mauša, & T. G. Grbac, "Co-evolutionary multi-population genetic programming for classification in software defect prediction: An empirical case study," *Applied Soft Computing*, vol. 55, pp. 331-351, 2017.
 - [18] Y. Jiang, B. Cukic, & Y. Ma, "Techniques for evaluating fault prediction models," *Empirical Software Engineering*, vol. 13(5), pp. 561-595, 2008.
 - [19] D. Rodriguez, I. Herraiz, R. Harrison, J. Dolado, & J. C. Riquelme, "Preliminary comparison of techniques for dealing with imbalance in software defect prediction," In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ACM, pp. 43, May 2014.
 - [20] G. Abaei, & A. Selamat, "A survey on software fault detection based on different prediction approaches," *Vietnam Journal of Computer Science*, vol. 1(2), pp. 79-95, 2014.
 - [21] I. Arora, & A. Saha, "Software Defect Prediction: A Comparison Between Artificial Neural Network and Support Vector Machine," In *Advanced Computing and Communication Technologies*, Springer, Singapore, pp. 51-61, 2018.
 - [22] Y. Peng, G. Kou, G. Wang, H. Wang, & F. I. Ko, "Empirical evaluation of classifiers for software risk management," *International Journal of Information Technology & Decision Making*, vol. 8(04), pp. 749-767, 2009.
 - [23] L. Chen, B. Fang, Z. Shang, & Y. Tang, "Tackling class overlap and imbalance problems in software defect prediction," *Software Quality Journal*, vol. 26(1), pp. 97-125, 2018.
 - [24] M. Benaddy, & M. Wakrim, "Simulated annealing neural network for software failure prediction," *International Journal of Software Engineering and Its Applications*, vol. 6(4), pp. 35-46, 2012.
 - [25] P. Zhang, & Y. T. Chang, "Software fault prediction based on grey neural network," In *Natural Computation (ICNC)*, 2012 Eighth International Conference on IEEE, pp. 466-469, May 2012.
 - [26] Y. Kamei, T. Fukushima, S. McIntosh, K. Yamashita, N. Ubayashi, & A. E. Hassan, "Studying just-in-time defect prediction using cross-project models," *Empirical Software Engineering*, vol. 21(5), pp. 2072-2106, 2016.
 - [27] P. He, B. Li, X. Liu, J. Chen, & Y. Ma, "An empirical study on software defect prediction with a simplified metric set," *Information and Software Technology*, vol. 59, pp. 170-190, 2015.
 - [28] X. Y. Jing, S. Ying, Z. W. Zhang, S. S. Wu, & J. Liu, "Dictionary learning based software defect prediction," In *Proceedings of the 36th International Conference on Software Engineering*, ACM, pp. 414-423, May 2014.
 - [29] Y. Peng, G. Kou, G. Wang, W. Wu, & Y. Shi, "Ensemble of software defect predictors: an AHP-based evaluation method," *International Journal of Information Technology & Decision Making*, vol. 10(01), pp. 187-206, 2011.
 - [30] I. Arora, & A. Saha, "Software Defect Prediction: A Comparison Between Artificial Neural Network and Support Vector Machine," In *Advanced Computing and Communication Technologies*, Springer, Singapore, pp. 51-61, 2018.
 - [31] X. Xia, D. Lo, E. Shihab, X. Wang, & X. Yang, "Elblocker: Predicting blocking bugs with ensemble imbalance learning," *Information and Software Technology*, vol. 61, pp. 93-106, 2015.
 - [32] Z. Sun, Q. Song, & X. Zhu, "Using coding-based ensemble learning to improve software defect prediction," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42(6), pp. 1806-1817, 2012.
 - [33] Q. Song, Z. Jia, M. Shepperd, S. Ying, & J. Liu, "A general software defect-proneness prediction framework," *IEEE Transactions on Software Engineering*, vol. 37(3), pp. 356-370, 2011.
 - [34] K. E. Bennin, J. Keung, A. Monden, P. Phannachitta, & S. Mensah, "The significant effects of data sampling approaches on software defect prioritization and classification," In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, IEEE Press, pp. 364-373, November 2017.
 - [35] S. Wang, & X. Yao, "Using class imbalance learning for software defect prediction," *IEEE Transactions on Reliability*, vol. 62(2), pp. 434-443, 2013.